概要

基本情報

タイトル　　：三目並べ
ジャンル　　：テーブルゲーム
制作期間　　：2026/4/7～4/22
制作人数　　：1人

使用言語　　：C++
使用ツール　：VisualStudio2026,
　　DxLib
対応プラットフォーム：Windows

制作意図

２年次にUnityでチーム制作を行ったときに自作で機能を作った経験から基礎的なプログラミング力を上げたいと思い、テーブルゲームを制作しようと考えました。

ゲーム内容

マウスカーソルをコマを置きたいマスまで運び、左クリックで置くことができます。
先に自分のコマを縦か横か斜めに３つそろえたほうの勝ちになります。そろわなかったら引き分けです。

アピールポイント

1. クラス分けによる責務の分離
2. 名前空間の活用
3. コマの設置処理を表示切り替えにする
4. 勝利判定のパターン化

1. クラス分けによる責務の分離

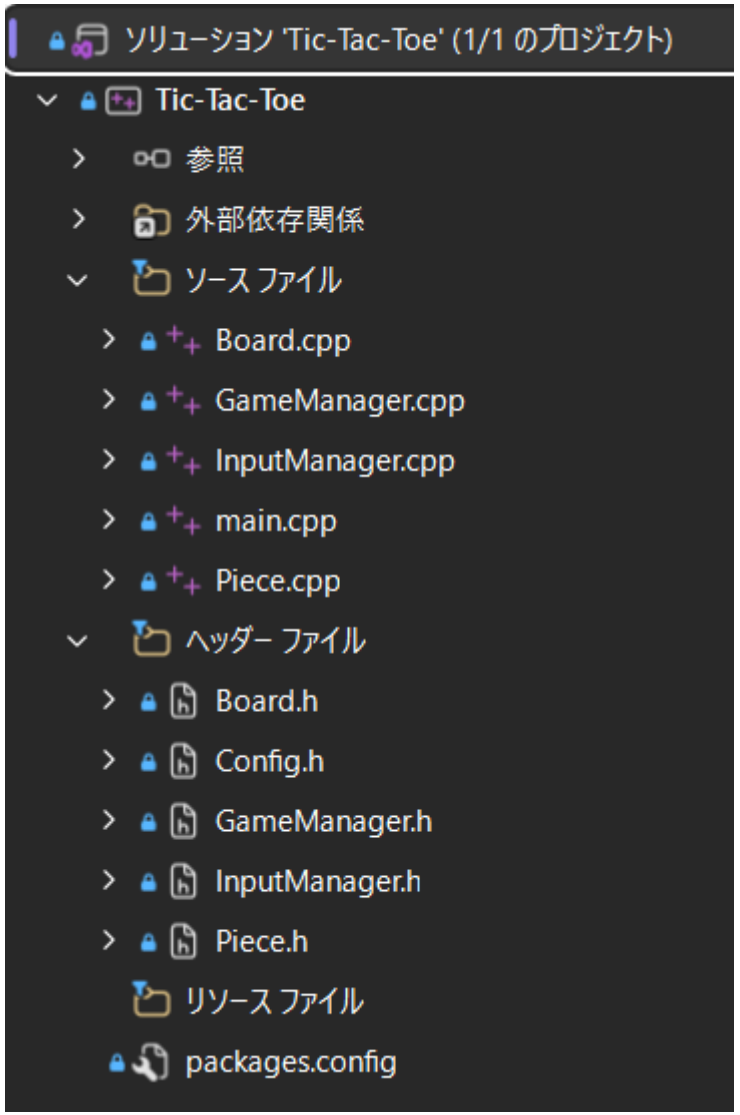
プロジェクトのファイル構成を以下のようにして、管理や修正がしやすいように工夫しました。

①InputManager
入力の管理をします。主に左クリックの状態を更新・保持しています。

②GameManager
ゲームの進行管理をします。主にゲーム全体の更新や描画を行います。

③Board
ボードの処理を担当します。主にコマの設置や勝利判定を行います。

④Piece
コマの処理を担当します。主に状態の受け渡しを行います。



2. 名前空間の活用

名前空間を使って似た定数を見やすく、使いやすくできるようにしました。

主な使用用途
①ゲームの状態管理
ゲーム内で使用する複数の状態をどこからでも使えるようにする目的で作成しました。

②ウィンドウの初期値
幅や高さを基準にボードやコマのサイズを決めるため、定数として定義しました。

該当スクリプト
Config.h

```
// ゲーム全体の設定
namespace Game
{
    // ゲームの状態
    enum State
    {
        TITLE,          // タイトル
        GAME_START,     // ゲームスタート
        GAME_SELECT,    // ゲーム中
        GAME_END,       // ゲーム終了
        RESULT,         // リザルト
    };
    // リザルトの種類
    enum ResultKind
    {
        FIRST_WIN,      // 先手の勝ち
        SECOND_WIN,     // 後手の勝ち
        DRAW,           // 引き分け
        CONTINUE,       // 勝敗なし
    };
    // 手番
    enum Turn
    {
        FIRST,      // 先手
        SECOND,     // 後手
        ALL         // すべて
    };
}
```

```
        Config::Game::ResultKind result = board.GetCheckResult();
        switch (result)
        {
        case Config::Game::FIRST_WIN:
            resultMessage = "１の勝ち";
            break;
        case Config::Game::SECOND_WIN:
            resultMessage = "２の勝ち";
            break;
        case Config::Game::DRAW:
            resultMessage = "引き分け";
            break;
        case Config::Game::CONTINUE:
        default:
            break;
        }

        // 結果を表示
        return;
    }
```

３.コマの設置処理を表示切り替えにする

コマの設置処理を見やすくするために
新しいコマの生成はボードの初期化で行い、
設置処理はコマの表示を切り替えるようにしました。

①Boardの初期化でSetPieceInitPos関数でコマを
「非表示」にして９つ設置する。

②Boardがコマを設置するときに指定のコマのみを
「表示」にする。

③コマの描画処理では「表示」のときだけ描画する。

該当箇所
Board.cpp → SetPieceInitPos関数, PutPiece関数
Piece.cpp → SetIsVisible関数, Draw関数

```
74      bool Board::PutPiece(int posX, int posY, Config::Game::Turn turn)
75      {
76          // ボード内の座標が盤の外なら、falseを返す
77          int num = GetSpaceNumber(posX, posY);
78          if (num == -1) return false;
79
80          // すでに置かれているなら、falseを返す
81          if (pieces[num].GetIsVisible()) return false;
82
83          // そうでないなら、コマを置いてtrueを返す
84          if(turn == Config::Game::Turn::FIRST) pieces[num].SetIsVisible(true, true);
85          if(turn == Config::Game::Turn::SECOND) pieces[num].SetIsVisible(true, false);
86
87          return true;
88      }
```

```
38      /// <summary>
39      /// 表示を切り替えて、対応するコマの色を設定する
40      /// </summary>
41      /// <param name="state">切り替えたい状態</param>
42      /// <param name="isFirst">先手かどうか</param>
43      void Piece::SetIsVisible(bool state, bool isFirst)
```

```
17      /// <summary>
18      /// 描画処理
19      /// </summary>
20      void Piece::Draw() const
21      {
22          if (isVisible == false) return;
23
24          DrawCircle(x, y, radius, color, TRUE);
25      }
```

４. 勝利判定のパターン化

三目並べは勝利パターンが決まっているため、
勝利する組み合わせを二次元配列で持つことで処理を簡単にしました。

三目並べのマスは以下のように番号で保存しています

0	1	2
3	4	5
6	7	8

赤色の列が揃っているかは
０と３と６のコマが
「表示」かつ「同じコマ」か
を調べることで判定できます。

該当スクリプト
board.cpp → Boardコンストラクタ，CheckBoard関数

```
17          checkPatterns(
18              {0, 3, 6},      // 左列
19              {1, 4, 7},      // 中列
20              {2, 5, 8},      // 右列
21              {0, 1, 2},      // 上行
22              {3, 4, 5},      // 中行
23              {6, 7, 8},      // 下行
24              {0, 4, 8},      // 斜め（左上から右下）
25              {2, 4, 6}       // 斜め（右上から左下）
26              )
```

```
147     // 縦・横・斜めのどれか一列が揃っているか判定をする
148     for (int i = 0; i < 8; i++)
149     {
150         // 表示されているか調べる
151         bool isVisible = true;
152         for (int j = 0; j < split; j++)
153         {
154             // 表示されていないコマがあるなら、falseにする
155             if (!pieces[checkPatterns[i][j]].GetIsVisible()) isVisible = false;
156         }
157
158         // １つでも表示されていないなら、以降の処理をとばす
159         if (isVisible == false) continue;
160
161         // 揃っているか調べる
162         bool isFirstPiece = pieces[checkPatterns[i][0]].GetIsFirstPiece();
163         bool isMatch = true;
164         for (int j = 0; j < split - 1; j++)
165         {
166             // １つでも異なるコマがあるなら、falseにする
167             if (isFirstPiece ^ pieces[checkPatterns[i][j + 1]].GetIsFirstPiece()) isMatch = false;
168         }
169
170         // 揃っていてかつ表示されているなら、結果を代入して終了する
171         if (isMatch && isVisible)
172         {
173             if (isFirstPiece) checkResult = Config::Game::ResultKind::FIRST_WIN;
174             else checkResult = Config::Game::ResultKind::SECOND_WIN;
175             return;
176         }
177     }
```

反省

反省点

①クラスごとの責務を曖昧にしたまま開発を進めたことで、勝利判定が複雑化してしまったこと。
②ゲームの状態にタイトルとリザルトを用意したが、状態遷移処理が作れなかったため使わなかったこと。
③名前空間の理解が曖昧だったため、不適切なものも名前空間に定義してしまい管理がしづらかったこと。
まとめ
クラスの責務を明確にしてから開発を進めなかったことで設計が複雑化して、進行が難しかったこと。

改善点・学び

①制作を開始する前に、クラスごとの役割や責務を決めてから制作を開始すること。
②ゲーム全体の進行管理にはステートパターンを用いた状態遷移処理を作成すること。
③名前空間を使う際には、それがプロジェクト全体で使いたいものなのかを考えてから使うこと。
まとめ
制作開始前に設計を考えることで制作を円滑に進めることができる。